

TI_86_OLED_Statusanzeige (zweiter Teil von TI_85)

Aufbau

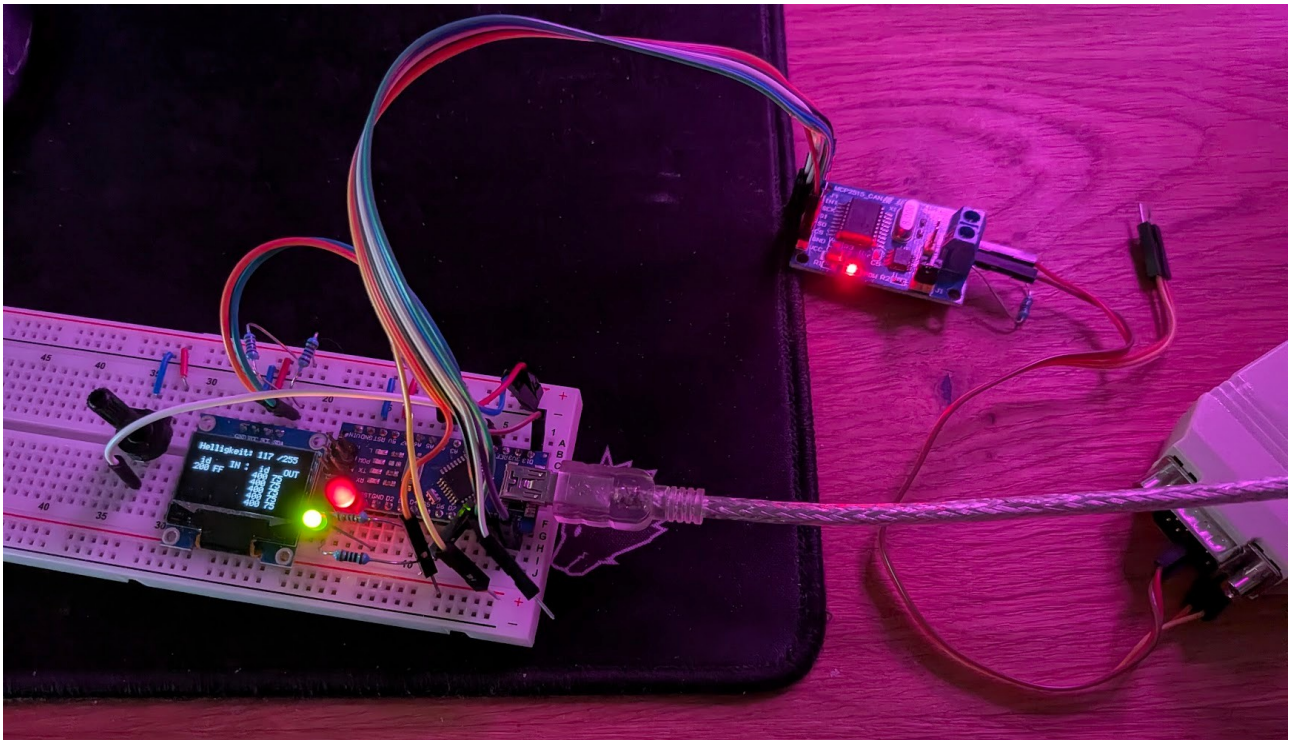


Abbildung 1: Gesamtübersicht

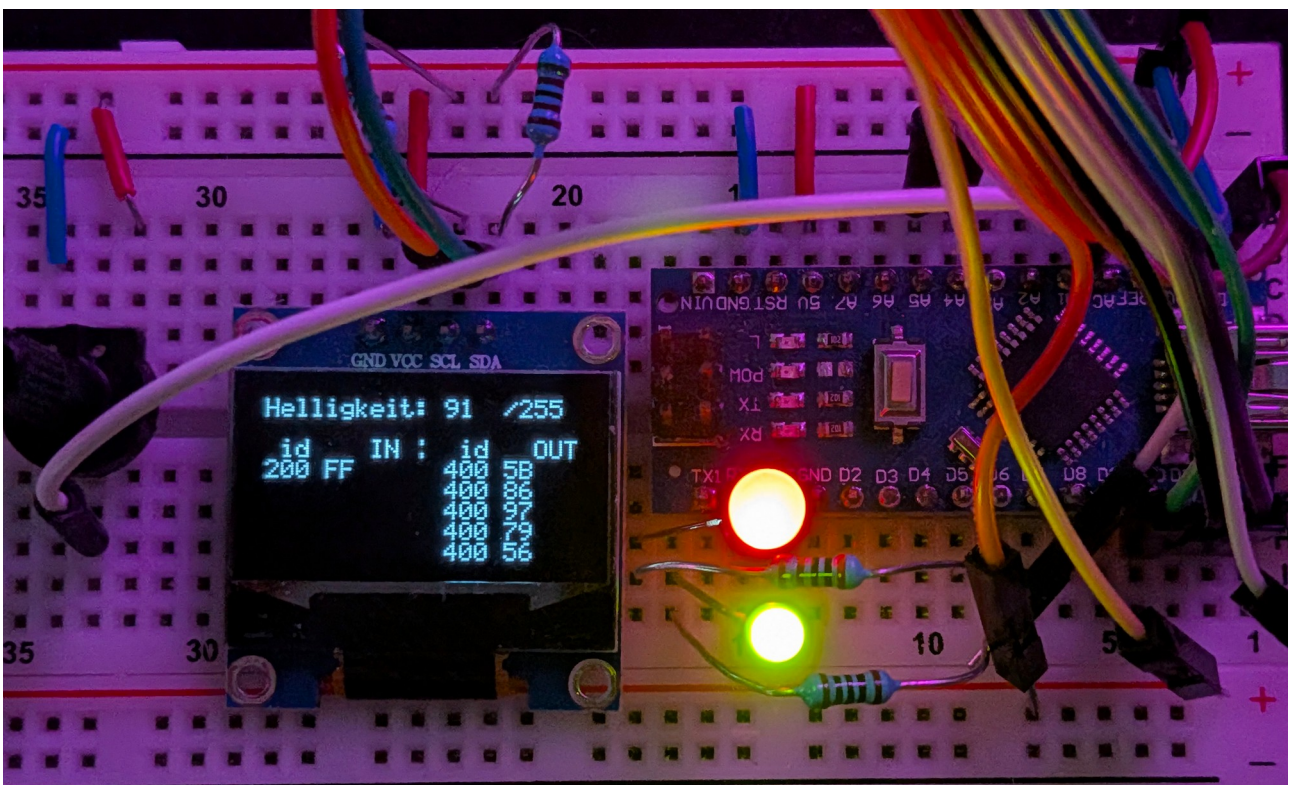


Abbildung 2: Aufbau auf Breadboard

In Abbildung 1 sieht man rechts den PCAN (CAN-Sniffer) und in der Mitte ein CAN-Modul mit dem MCP2515 (Controller) und TJA1050 (Transceiver).

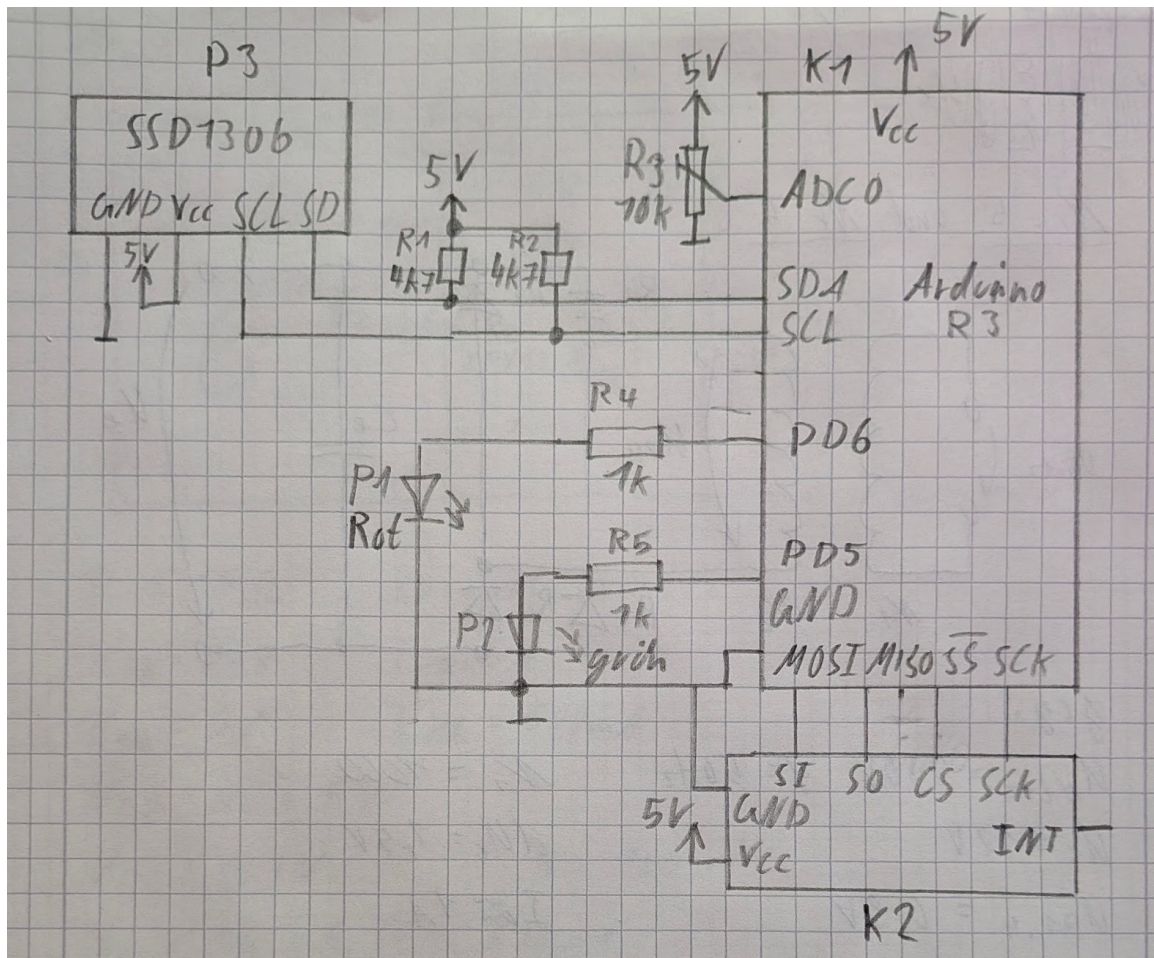


Abbildung 3: Schaltplan

	ID	RTR	Helligkeitswert
Datentyp	11 Bit	1 Bit	8 Bit (von 0 bis 255)
Beispiel	2FA	0	AA (170)

Tabelle 1: CAN-Bus-Nachrichtformat

Der hier gezeigte Aufbau ist zwar mit einem Arduino (ATmega328P), allerdings funktioniert der Code auch mit dem AT90CAN32. Wenn möglich baue ich erst mit dem Arduino auf, um eine Beschädigung des AT90CAN32 zu vermeiden. Der AT90CAN32 hat einen eingebauten CAN-Controller dadurch könnte man das CAN-Modul weglassen.

Die einzelnen LEDs zeigen die erfasste Helligkeit an. Dabei ist die grüne LED für die erhaltene Helligkeit und die rote für die lokale Helligkeit.

Quellcode

CMakeLists.txt

```
cmake_minimum_required(VERSION 4.1)
project(TI_x C)

set(CMAKE_C_STANDARD 99)
set(CMAKE_C_EXTENSIONS OFF)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

set(MCU atmega328p)
set(F_CPU 16000000UL)
set(PROGRAMMER arduino)
set(PORT /dev/ttyUSB0 CACHE STRING "tty port")

# Generic: no OS
set(CMAKE_SYSTEM_NAME Generic)
set(CMAKE_C_COMPILER avr-gcc)

# Compiler flags
add_definitions(-DF_CPU=${F_CPU})
set(CMAKE_C_FLAGS "-mmcu=${MCU} -Os")
set(CMAKE_EXE_LINKER_FLAGS "-mmcu=${MCU}")

add_executable(${PROJECT_NAME}.elf
    src/main.c
)

add_subdirectory(lib)
target_link_libraries(${PROJECT_NAME}.elf PRIVATE usart can oled-display)

# convert ELF to IHEX
add_custom_command(
    OUTPUT ${PROJECT_NAME}.hex
    DEPENDS ${PROJECT_NAME}.elf
    COMMAND avr-objcopy -O ihex -R .eeprom ${PROJECT_NAME}.elf $
${PROJECT_NAME}.hex
)

add_custom_target(hex
    DEPENDS ${PROJECT_NAME}.hex
)

add_custom_target(upload
    ALL
    DEPENDS hex
    COMMAND avrdude -p ${MCU} -P ${PORT} -c ${PROGRAMMER} -U flash:w:$
${PROJECT_NAME}.hex
)
```

Kompilieren mit:

```
$ cmake -B build
$ cmake --build build
```

main.c

```
#include "can.h"
#include "lcd.h"
#include "usart.h"
#include <avr/io.h>
#include <avr/pgmspace.h>
#include <stdbool.h>
#include <stdint.h>
#include <stdlib.h>
#include <util/delay.h>

static const uint16_t ID_DEAN = 0x200;
static const uint16_t ID_HENRY = 0x400;

static const uint8_t SEND_INPUT = ADC0D;
static const uint8_t RECEIVE_LED = PD5;
static const uint8_t SEND_LED = PD6;

#define RECEIVE_LED_PWM OCR0A
#define SEND_LED_PWM OCR0B

/// analog to digital converter init
void adc_init(const uint8_t adc_pin) {
    // use Vcc as reference voltage, use ADC0
    ADMUX = 1 << REFS0 | adc_pin;
    // enable ADC, F_CPU/128 prescaling so average can be measured
    ADCSRA = 1 << ADEN | 1 << ADPS2 | 1 << ADPS1 | 1 << ADPS0;
    // disable digital input at pin ADC0
    DIDR0 = ADC0D;
}

/// read analog to digital converter pin, waits until finished
uint16_t adc_read_sync(void) {
    // start conversion
    ADCSRA |= 1 << ADSC;

    // wait for ADC
    while (ADCSRA >> ADSC & 1)
        ;

    return ADCL | ADCH << 8;
}

void pwm_init(void) {
    // use timer0, Clear OC0A (PD6) on compare match, set OC0A at BOTTOM,
    // (non-inverting mode), fast PWM
    TCCR0A = 1 << COM0A1 | 1 << COM0B1 | 1 << WGM01 | 1 << WGM00;
    // F_CPU without prescaling
    TCCR0B = 1 << CS00;
}
```

Mehr Information über die verwendeten Register (z. B. ADMUX, TCCR0A) lassen sich im Datenblatt des ATmega328P finden.

```

/// update brightness value on lcd
void update_brightness_lcd(const uint8_t value) {
    char value_str[4]; // 3 digits + \0

    utoa(value, value_str, 10);

    lcd_gotoxy(12, 0); // x offset of "Helligkeit: "
    lcd_puts("    "); // clear value

    lcd_gotoxy(12, 0); // x offset of "Helligkeit: "
    lcd_puts(value_str);
}

/// @return false falls die Nachricht nicht verschickt werden konnte,
/// ansonsten der Code des Puffes in dem die Nachricht gespeichert wurde
uint8_t send_brightness_can(const uint8_t brightness,
                           const uint16_t target_id) {
    can_t msg = {
        .id = target_id,
        .flags.rtr = 0,
        .length = 1,
        .data = {brightness},
    };

    return can_send_message(&msg);
}

```

update_brightness_lcd

Zu beginn wird „Helligkeit: “ auf das Display geschrieben. Anschließend an den Text wird der Wert geschrieben.

```

/// @param hex 4 bit value in lower nibble
char hex_to_ascii(uint8_t value) {
    value &= 0x0F;

    if (value <= 9) {
        return '0' + value;
    } else {
        return 'A' + (value - 10);
    }
}

#define LINE_LEN 7
#define LINE_COUNT 5
#define BUFFER_LEN (LINE_LEN * LINE_COUNT)

/// insert new message at top on LCD
/// @returns new buffer_pos
char *_put_message_lcd(char *const buffer, char *buffer_pos, const uint16_t id,
                      const uint8_t message, const uint8_t x_offset) {
    // warn: has to match buffer letters
    *buffer_pos++ = hex_to_ascii(id >> 8);
    *buffer_pos++ = hex_to_ascii(id >> 4);
    *buffer_pos++ = hex_to_ascii(id);
    *buffer_pos++ = ' ';
    *buffer_pos++ = hex_to_ascii(message >> 4);
    *buffer_pos++ = hex_to_ascii(message);
    *buffer_pos++ = '\0';

    // reverse iterator, starting with current
    char *buffer_reverse_it = buffer_pos - LINE_LEN;

    if (buffer_pos == buffer + BUFFER_LEN) {
        buffer_pos = buffer;
    }

    for (uint8_t line = 0; line < LINE_COUNT; line++) {
        lcd_gotoxy(x_offset, 3 + line); // starting at line 3
        lcd_puts(buffer_reverse_it);

        buffer_reverse_it -= LINE_LEN;

        if (buffer_reverse_it < buffer) {
            buffer_reverse_it = buffer + BUFFER_LEN - LINE_LEN; // last line
        }
    }

    return buffer_pos;
}

```

Bei der unteren Funktionen wird eine CAN-Bus Nachricht entweder links oder rechts angezeigt. Damit neue Nachrichten alte nicht überschreiben werden die Nachrichten gepuffert. Bei der Ausgabe wird die neueste Nachricht zuerst angezeigt.

```

/// insert new message at top on LCD in ingoing column
void put_message_lcd_in(const uint16_t id, const uint8_t message) {
    static char buffer[LINE_LEN * LINE_COUNT];
    static char *buffer_pos = buffer;

    buffer_pos = _put_message_lcd(buffer, buffer_pos, id, message, 0);
}

/// insert new message at top on LCD in outgoing column
void put_message_lcd_out(const uint16_t id, const uint8_t message) {
    static char buffer[LINE_LEN * LINE_COUNT];
    static char *buffer_pos = buffer;

    buffer_pos = _put_message_lcd(buffer, buffer_pos, id, message, 12);
}

void init(void) {
    DDRD = 1 << RECEIVE_LED | 1 << SEND_LED | 0 << SEND_INPUT;

    uart0_init(BAUD_CALC(9600UL));
    uart0_puts("TI_86_OLED_Statusanzeige\r\n");

    adc_init(ADC0D);
    pwm_init();
    lcd_init(LCD_DISP_ON);

    // Initialize MCP2515 with 250 kB/s because of 8 MHz crystal
    // actual bus speed is 125 kB/s
    // CS pin is configured in lib/can/include/config.h
    if (!can_init(BITRATE_250_KBPS)) {
        uart0_puts("error: can_init\r\n");

        while (1) {
        }
    }

    lcd_puts_p(PSTR("Helligkeit: "));

    lcd_gotoxy(15, 0); // "Helligkeit: xxx/255"
    lcd_puts_p(PSTR("/255"));

    lcd_gotoxy(0, 2);
    lcd_puts_p(PSTR(" id      IN | id      OUT"));
}

```

Jenachdem ob eine Nachricht rausgeht oder eingeht wird die jeweilige Funktion gewählt. Die gespaltete Nachrichtenanzeige auf dem Display beginnt ab Zeile drei, die Überschrift befindet sich in Zeile zwei.

```

int main(void) {
    init();

    while (1) {
        const uint8_t brightness_local = adc_read_sync() >> 2;

        uart0_puts("lokale Helligkeit: ");
        uart0_putuint(brightness_local);
        uart0_puts("\r\n");

        SEND_LED_PWM = brightness_local;
        update_brightness_lcd(brightness_local);
        put_message_lcd_out(ID_HENRY, brightness_local);

        uart0_puts("Send brightness over can\r\n");
        send_brightness_can(brightness_local, ID_HENRY);

        uart0_puts("wait for CAN message...\r\n");

        can_t received_msg;

        if (!can_get_message(&received_msg)) {
            _delay_ms(500); // optional: reduce message rate
            continue;
        }

        uart0_puts("received message from 0x");
        uart0_putc(hex_to_ascii(received_msg.id >> 8));
        uart0_puthex(received_msg.id);
        uart0_puts("\r\n");

        if (received_msg.id != ID_DEAN) {
            continue;
        }

        if (received_msg.length != 1) {
            uart0_puts("error: received CAN message is larger than 1\r\n");
            continue;
        }

        const uint8_t brightness_received = received_msg.data[0];

        uart0_puts("received brightness value: ");
        uart0_putuint(brightness_received);
        uart0_puts("\r\n");

        RECEIVE_LED_PWM = brightness_received;
        put_message_lcd_in(received_msg.id, brightness_received);
    }
}

void setup() { main(); }

void loop() {}

```

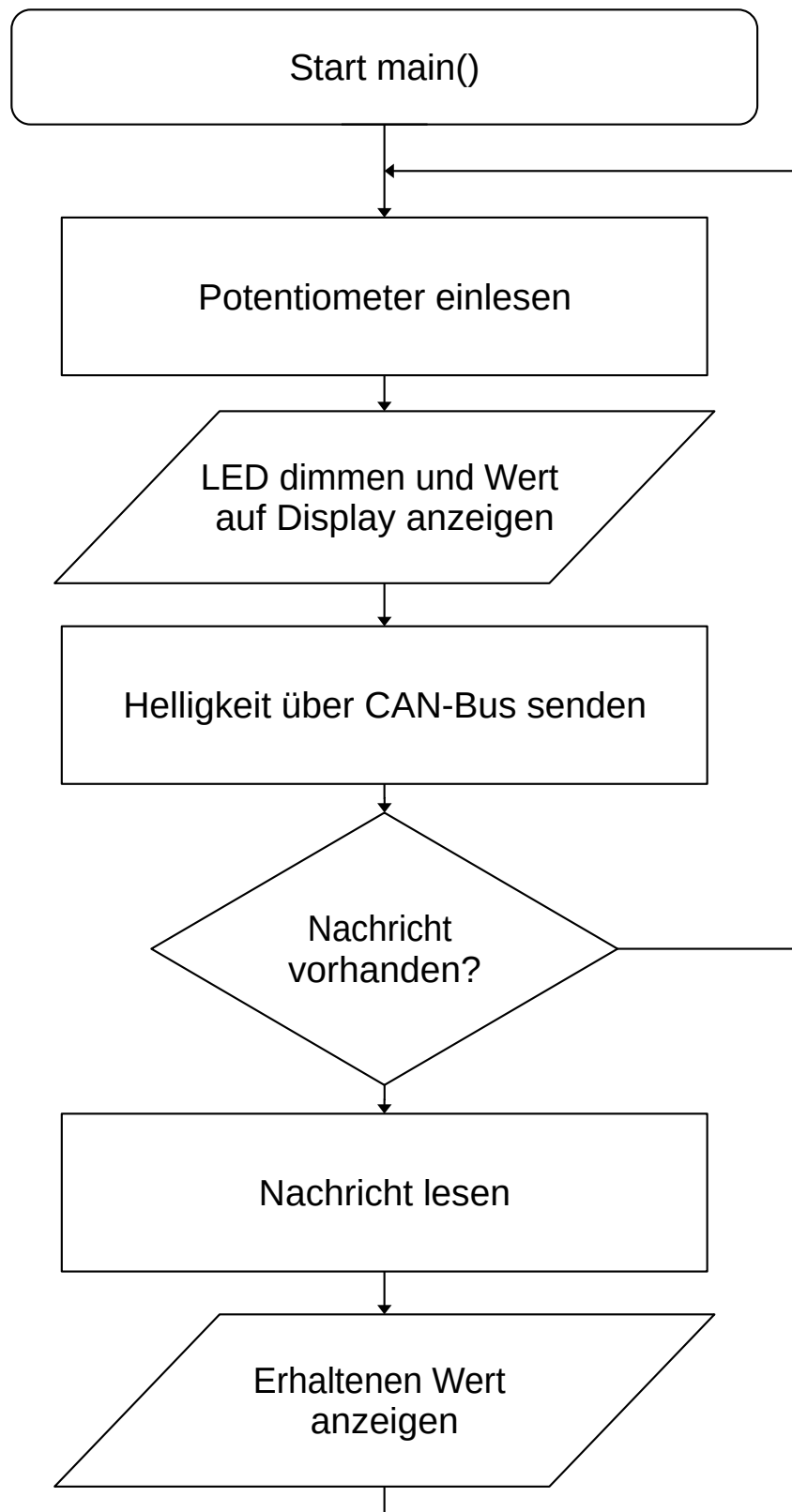



Abbildung 4: Programmübersicht